STUDENT MANAGEMENT SYSTEM

Software Engineering II — Backend Server Project

SOFTWARE REQUIREMENTS SPECIFICATION

QR-Based Student Attendance & Management System

A Spring Boot REST API with JWT Authentication and Role-Based Access Control

Project:	Student Management Server (SMS)
Module:	com.lechthemitch.sms
Framework:	Spring Boot 4.0.6 / Java 25
Database:	MySQL (via Docker Compose)
Document Version:	1.0
Date:	May 2026

CONFIDENTIAL — FOR ACADEMIC SUBMISSION ONLY

Table of Contents

1. System Overview
2. System Architecture
3. Functional Requirements
4. Non-Functional Requirements
5. Use Case Diagram
6. Activity Diagrams
    1. User Authentication
    2. Forced Account Update
    3. QR Code Attendance Scanning
    4. Student Registration
    5. Generic CRUD Operations
7. Sequence Diagrams
    1. Login Flow
    2. JWT Filter Chain
    3. QR Attendance Scan
    4. Student Creation with Auto-Parent
8. Class Diagram
9. Entity Relationship Diagram (ERD)
10. API Reference
11. Module Descriptions
12. Database Design
13. Assumptions & Constraints
14. Glossary

1. System Overview

1.1 Purpose

This document constitutes the Software Requirements Specification (SRS) for the **Student Management System (SMS)**, a RESTful backend server developed with Spring Boot. The system manages student enrollments, parent accounts, academic session scheduling, hall assignments, and QR-code-driven attendance tracking within an educational institution. All documentation herein is derived strictly from static analysis of the submitted source code.

1.2 Scope

The SMS backend exposes a secured HTTP API consumed by a client application (web or mobile). It handles the full lifecycle of users across four distinct roles (Administrator, Student, Parent, Teaching Assistant), enforces JWT-based stateless authentication, and provides a QR code scanning workflow to record attendance. The system does *not* include a frontend; all interfaces are REST endpoints.

1.3 Project Identification

Attribute	Value
Application Name	Student Management Server
Maven Artifact ID	Student_Managment_Server
Base Package	com.lechthemitch.sms
Spring Boot Version	4.0.6
Java Version	25 (with Kotlin 2.2.0 support)
Persistence	Spring Data JPA / Hibernate (DDL auto: create)
Database	MySQL 8 (Docker Compose, port 8081 mapped to 3306)
Auth Mechanism	JWT (HMAC-SHA256, jjwt 0.13.0)
QR Code Library	Google ZXing 3.5.3
Password Encoding	BCrypt

1.4 Intended Audience

- Software engineers and reviewers evaluating the codebase
- Academic assessors judging the SE II project submission
- Future maintainers extending the system
- QA engineers designing test plans

1.5 System Context

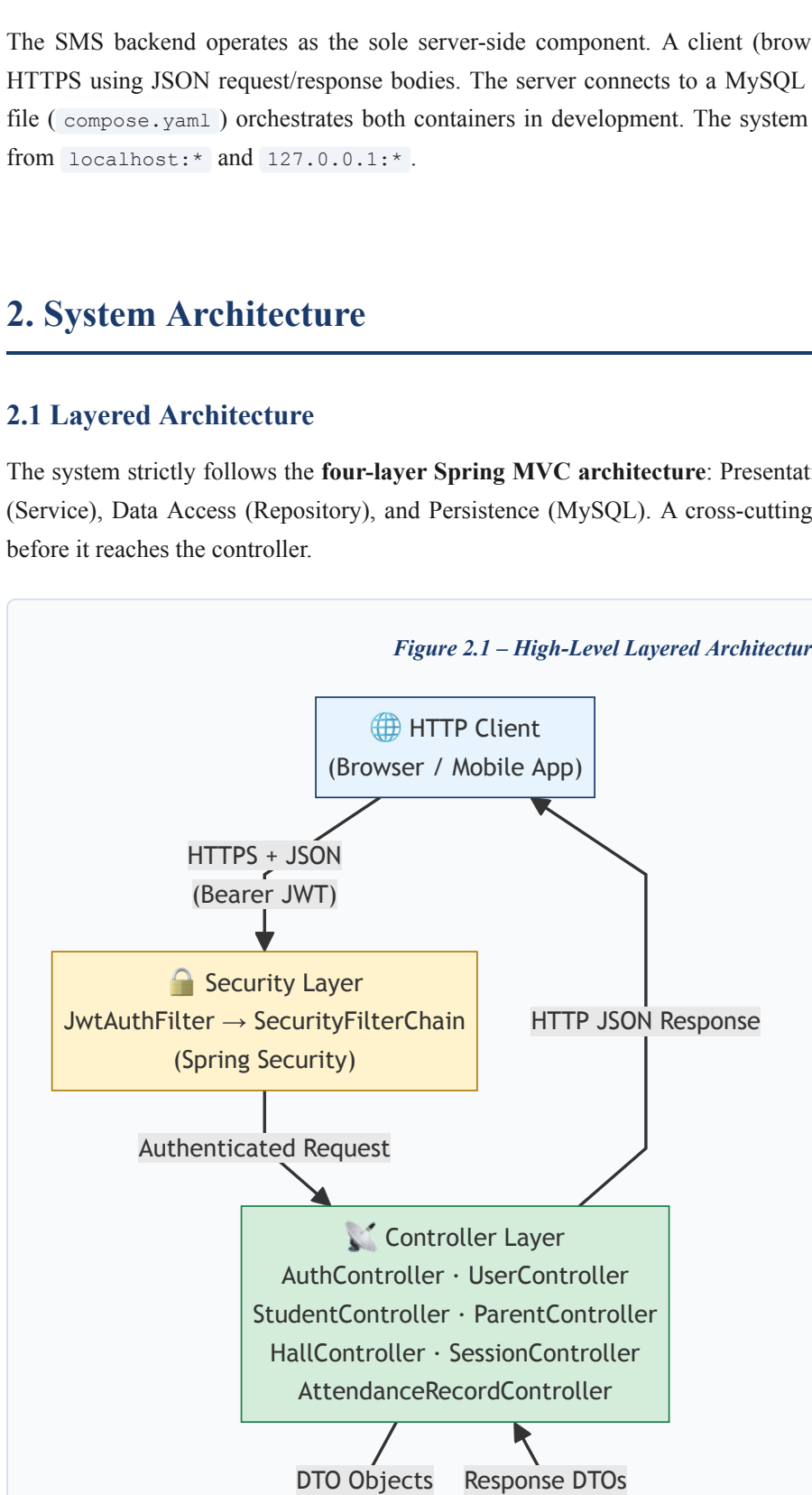
The SMS backend operates as the sole server-side component. A client (browser or mobile app) communicates over HTTPS using JSON request/response bodies. The server connects to a MySQL database container. A Docker Compose file (`compose.yaml`) orchestrates both containers in development. The system is CORS-configured to accept requests from `localhost:*` and `127.0.0.1:*`.

2. System Architecture

2.1 Layered Architecture

The system strictly follows the **four-layer Spring MVC architecture**: Presentation (REST Controllers), Business Logic (Service), Data Access (Repository), and Persistence (MySQL). A cross-cutting Security layer intercepts every request before it reaches the controller.

Figure 2.1 – High-Level Layered Architecture



HTTP Client
(Browser / Mobile App)
HTTPS + JSON
(Bearer JWT)
HTTP JSON Response
Security Layer
JwtAuthFilter → SecurityFilterChain
(Spring Security)
Authenticated Request
Controller Layer
AuthController · UserController
StudentController · ParentController
HallController · SessionController
AttendanceRecordController
DTO Objects
Response DTOs
Service Layer
AuthService · JwtService · QrCodeService
StudentServiceImpl · ParentServiceImpl
HallServiceImpl · SessionServiceImpl
AttendanceRecordServiceImpl · UserServiceImpl
JPA Entities
Entity Objects
Repository Layer
UserRepository · StudentRepository
ParentRepository · HallRepository
SessionRepository · AttendanceRecordRepository
RoleRepository · PermissionRepository
SQL
ResultSet
MySQL Database
SE2 Schema

2.2 Security Architecture

Every incoming HTTP request passes through the `JwtAuthFilter` (a `OncePerRequestFilter`) before reaching any controller. The filter extracts the Bearer token, validates it via `JwtService`, and populates the Spring Security `SecurityContext`. Three endpoints are publicly accessible without a token: `/login`, `/register`, and `/complete-forced-account-update`. All other endpoints require a valid JWT and additionally enforce method-level `@PreAuthorize` rules.

2.3 Technology Stack

Layer	Technology	Purpose
HTTP Server	Spring Boot Web MVC	REST API handling
Security	Spring Security 6 + JJWT	Auth filter, JWT generation/validation
Persistence	Spring Data JPA + Hibernate	ORM, schema generation
Database	MySQL 8 (Docker)	Relational data storage
Validation	Jakarta Bean Validation	DTO constraint enforcement
QR Code	Google ZXing 3.5.3	PNG QR code generation
Boilerplate	Lombok	Getter/Setter/Builder generation
Build	Maven + Spring Boot Maven Plugin	Build, packaging, Docker
Containerization	Docker + Compose	Local development environment

3. Functional Requirements

3.1 Authentication & Account Management

ID	Requirement
FR-AUTH-01	The system shall allow any client to register a new user account by providing first name, last name, email, password, phone number, role, and optional direct permissions.
FR-AUTH-02	The system shall reject registration if the supplied email already exists.
FR-AUTH-03	The system shall allow login using either an email address or a phone number combined with a password.
FR-AUTH-04	Upon successful authentication, the system shall return a signed JWT containing the user's role and permissions as claims.
FR-AUTH-05	The system shall detect accounts flagged for forced update (forcePasswordChange, forceEmailChange, or email starting with "parent_") and block normal login, redirecting users to the forced-update endpoint.
FR-AUTH-06	The system shall allow a forced account update by verifying the current password, then updating the email and password, clearing forced-update flags, and issuing a fresh JWT.
FR-AUTH-07	Passwords shall be stored as BCrypt hashes; plain-text passwords shall never be persisted.
FR-AUTH-08	JWT tokens shall embed role claims (prefixed ROLE_) and permission claims and shall expire after the configured duration (default 10 hours).

3.2 User Management

ID	Requirement
FR-USR-01	Administrators shall be able to retrieve a list of all users.
FR-USR-02	Any authenticated user shall be able to retrieve their own profile, including merged role and direct permissions, linked parentId, and studentId.

3.3 Student Management

ID	Requirement
FR-STU-01	Authorized users shall be able to create a Student profile linked to an existing User account.
FR-STU-02	During student creation, if no parentId is supplied, the system shall automatically locate or create a Parent user account (email prefixed "parent_", password set to the phone number, forced update flags enabled).
FR-STU-03	Each student shall be assigned a cryptographically unique URL-safe Base64 QR code token (24 random bytes) at creation time.
FR-STU-04	The system shall expose a PNG image endpoint (`/api/students/{id}/qr`) that renders the student's QR code at a configurable resolution.
FR-STU-05	Students with existing attendance records shall not be deletable.
FR-STU-06	A student's userId shall be immutable after creation.
FR-STU-07	Students can be retrieved individually, collectively, or filtered by parentId.

3.4 Parent Management

ID	Requirement
FR-PAR-01	Admins and Teaching Assistants shall be able to create, list, and delete parent records.
FR-PAR-02	A parent shall be able to view their own record and their children's records; no cross-parent access is permitted.
FR-PAR-03	Admins, Teaching Assistants, or the parent themselves may update a parent record.

3.5 Hall Management

ID	Requirement
FR-HALL-01	Only Administrators may create, update, or delete halls.
FR-HALL-02	Each hall has a unique name, optional location, a scheduled session_time, and an optional link to a Session.
FR-HALL-03	Admins and Teaching Assistants may list and view all halls.

3.6 Session Management

ID	Requirement
FR-SES-01	Admins and Teaching Assistants shall be able to create, view, update sessions; only Admins may delete sessions.
FR-SES-02	A session consists of a title and optional description, and may have multiple halls and attendance records associated with it.

3.7 Attendance Record Management

ID	Requirement
FR-ATT-01	Attendance can be recorded via QR scan or manual entry, both restricted to Admins and Teaching Assistants.
FR-ATT-02	QR scan attendance shall resolve the student from the QR token, verify no duplicate attendance exists for that student+session pair, validate that the scanned hall belongs to the session, and timestamp the record.
FR-ATT-03	A student shall only have one attendance record per session (duplicate prevention).
FR-ATT-04	Attendance records can be filtered by student, session, or hall.
FR-ATT-05	A student may view their own attendance records; Admins and Teaching Assistants may view all.

3.8 Role-Based Access Control (RBAC)

Role	Capabilities Summary
ADMIN	Full system access: manage users, students, parents, halls, sessions, attendance
TEACHING_ASSISTANT	Create/view parents, students, sessions, halls; record attendance; no delete/admin operations
STUDENT	View own profile, view own attendance records
PARENT	View own profile, view own children, view own parent record

3.9 Data Seeding

On application startup, `DataSeeder` ensures baseline data exists: all Permission records, all Role records with default permission assignments, a default admin user, a default parent user, and a default student user with a linked parent and a generated QR code.

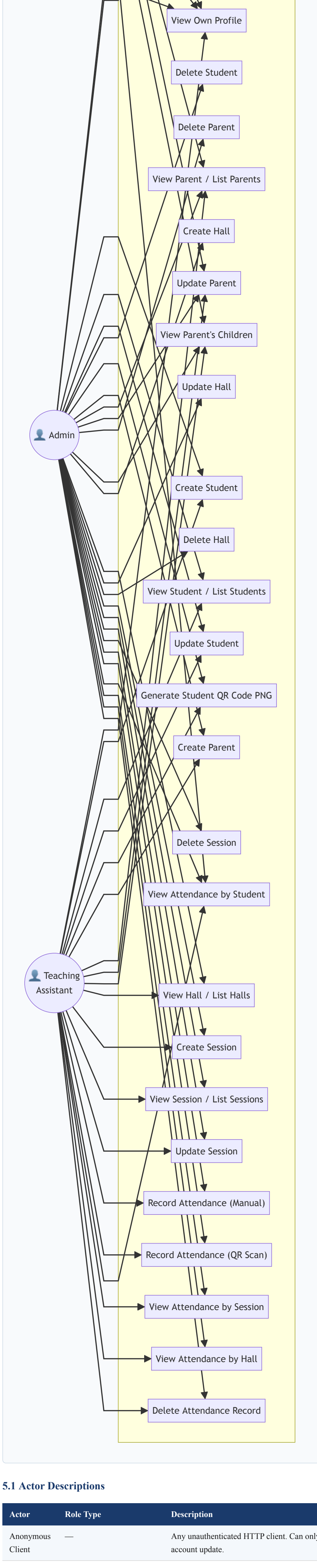
4. Non-Functional Requirements

ID	Category	Requirement
NFR-SEC-01	Security	All protected endpoints must validate a JWT signed with HMAC-SHA-256 before processing the request.
NFR-SEC-02	Security	Passwords must never be stored or returned in plaintext; BCrypt with Spring default strength is mandatory.
NFR-SEC-03	Security	Session management must be stateless (no server-side session storage).
NFR-SEC-04	Security	CSRF protection is disabled (stateless JWT removes CSRF risk); HTTP Basic Auth is disabled.
NFR-SEC-05	Security	CORS is restricted to localhost origins in the current configuration.
NFR-QR-01	Integrity	QR code tokens must be globally unique; generation must loop until uniqueness is confirmed.
NFR-VAL-01	Validation	All DTOs must be validated using Bean Validation annotations (@NotBlank, @Email, @Pattern, @Size) before service invocation.
NFR-ERR-01	Reliability	All exceptions must be handled by `GlobalExceptionHandler` returning consistent JSON error responses.
NFR-PERF-01	Performance	Role and permission fetching is configured as EAGER loading to minimise lazy-load round trips on authentication paths.
NFR-MAINT-01	Maintainability	Service interfaces must be separated from implementations (e.g., HallService / HallServiceImpl) to allow future substitution.
NFR-PORT-01	Portability	Database connection parameters are externalised via environment variables; the system must run via Docker Compose without code changes.
NFR-EXT-01	Extensibility	Permission and Role seeding is idempotent; new permissions can be added to the enum and DataSeeder without altering existing data.

5. Use Case Diagram

The following diagram identifies all system actors and their interactions with the major system use cases, derived from controller-level `@PreAuthorize` annotations and service logic.

Figure 5.1 – System Use Case Diagram



Student Management System
Anonymous Client
Parent
Student
Admin
Teaching Assistant
Register Account
Login
Complete Forced Account Update
List All Users
View Own Profile
Delete Student
Delete Parent
View Parent / List Parents
Create Hall
Update Parent
View Parent's Children
Update Hall
Create Student
Delete Hall
View Student / List Students
Update Student
Generate Student QR Code PNG
Create Parent
Delete Session
View Attendance by Student
View Hall / List Halls
Create Session
View Session / List Sessions
Update Session
Record Attendance (Manual)
Record Attendance (QR Scan)
View Attendance by Session
View Attendance by Hall
Delete Attendance Record

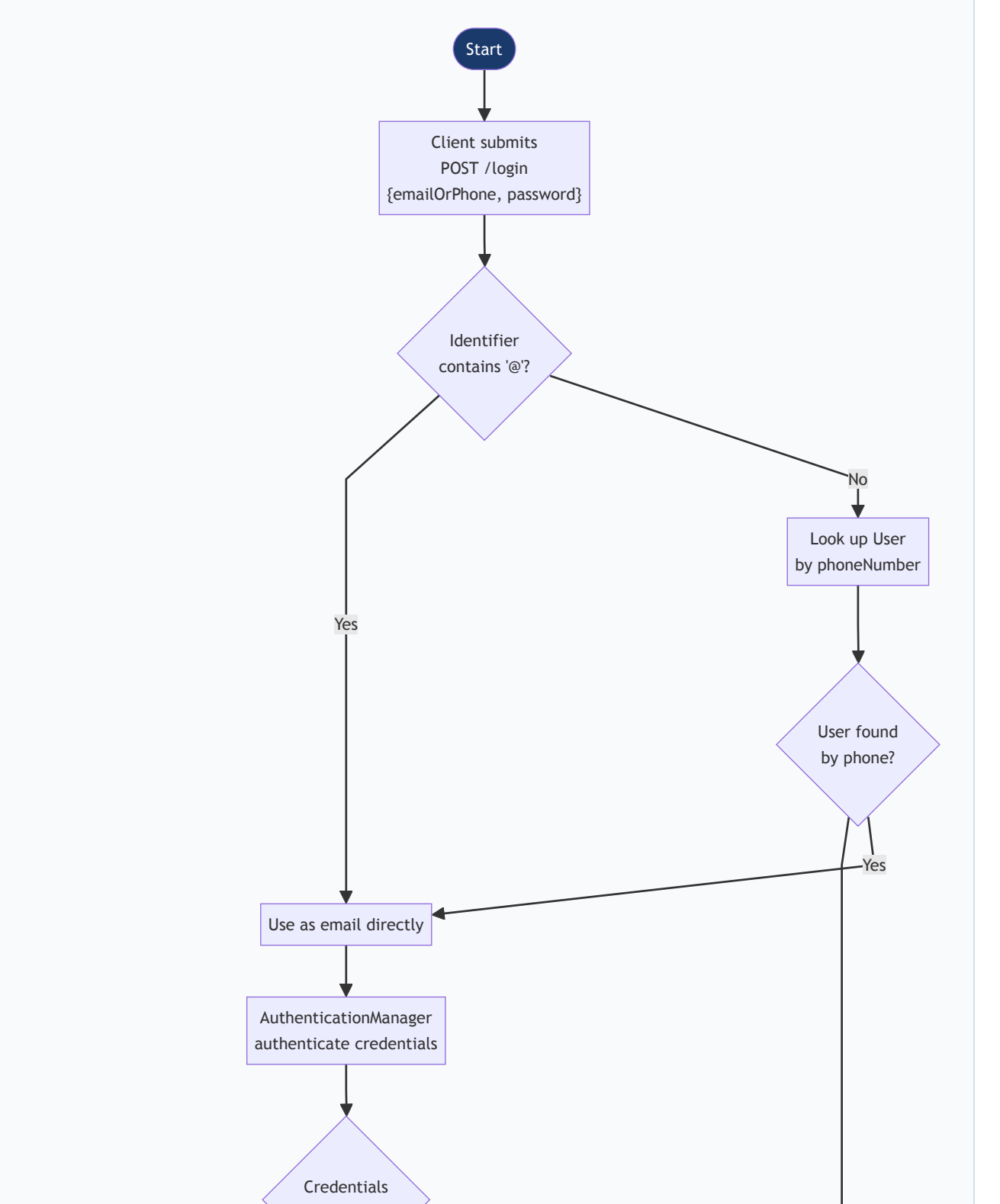
5.1 Actor Descriptions

Actor	Role Type	Description
Anonymous Client	—	Any unauthenticated HTTP client. Can only register, login, or complete forced account update.
Admin	ADMIN	Full system administrator. Has unrestricted access to all CRUD operations and management functions.
Teaching Assistant	TEACHING_ASSISTANT	Academic staff who can manage students, parents, sessions, and record attendance. Cannot create/delete halls or delete sessions.
Student	STUDENT	Enrolled student. Read-only access to own data and own attendance records.
Parent	PARENT	Parent/guardian. Can view and update their own record and view their children's data.

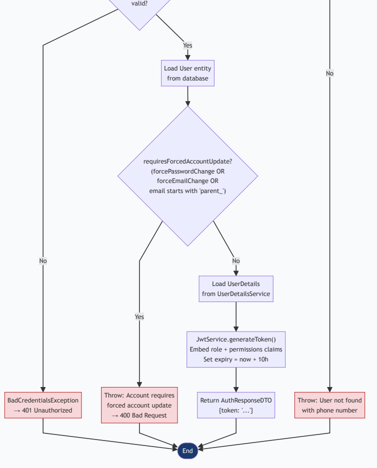
6. Activity Diagrams

6.1 User Authentication (Login)

Figure 6.1 – Login Activity Diagram

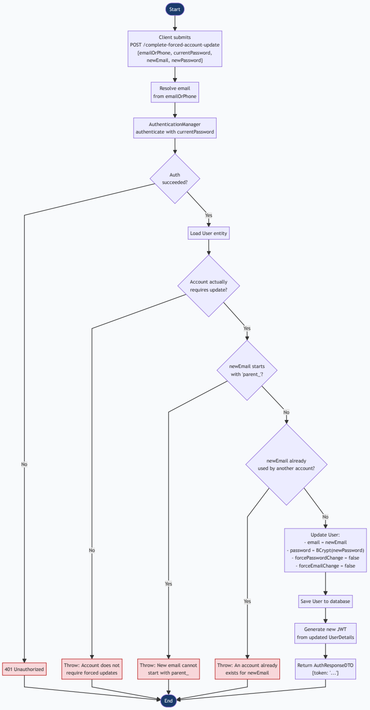


Start
Client submits
POST /login
{emailOrPhone, password}
Identifier
contains '@'?
No
Look up User
by phoneNumber
Yes
User found
by phone?
Yes
Use as email directly
AuthenticationManager
authenticate credentials
Credentials


valid?
Yes
No
Load User entity
from database
requiresForcedAccountUpdate?
(forcePasswordChange OR
forceEmailChange OR
email starts with 'parent_')
No
Yes
No
Load UserDetails
from UserDetailsService
JwtService.generateToken()
Embed role + permissions claims
Set expiry = now + 10h
BadCredentialsException
→ 401 Unauthorized
Throw: Account requires
forced account update
→ 400 Bad Request
Return AuthResponseDTO
{token: '...'}
Throw: User not found
with phone number
End

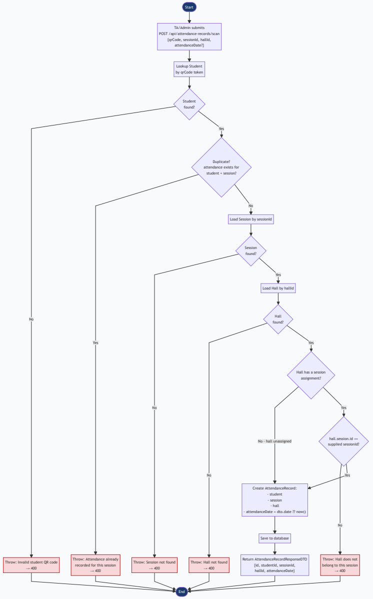
6.2 Forced Account Update

Figure 6.2 – Forced Account Update Activity Diagram


Start
Client submits
POST /complete-forced-account-update
{emailOrPhone, currentPassword,
newEmail, newPassword}
Resolve email
from emailOrPhone
AuthenticationManager
authenticate with currentPassword
Auth
succeeded?
Yes
Load User entity
Account actually
requires update?
Yes
newEmail starts
with 'parent_'?
No
newEmail already
used by another account?
No
Update User:
- email = newEmail
- password = BCrypt(newPassword)
- forcePasswordChange = false
- forceEmailChange = false
Save User to database
Generate new JWT
from updated UserDetails
No
No
Yes
Yes
401 Unauthorized
Throw: Account does not
require forced updates
Throw: New email cannot
start with parent_
Throw: An account already
exists for newEmail
Return AuthResponseDTO
{token: '...'}
End

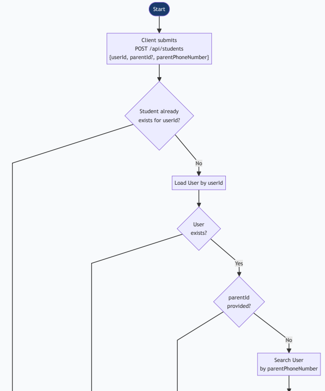
6.3 QR Code Attendance Scanning

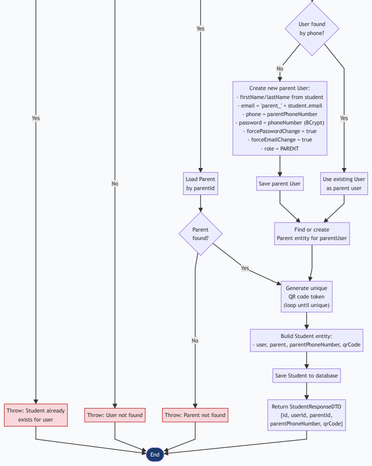
Figure 6.3 – QR Attendance Scan Activity Diagram


Start
TA/Admin submits
POST /api/attendance-records/scan
{qrCode, sessionId, hallId,
attendanceDate?}
Lookup Student
by qrCode token
Student
found?
Yes
No
Duplicate?
attendance exists for
student + session?
No
Yes
Load Session by sessionId
Session
found?
Yes
No
Load Hall by hallId
Hall
found?
Yes
No
Hall has a session
assignment?
Yes
No - hall unassigned
hall.session.id ==
supplied sessionId?
Yes
No
Create AttendanceRecord:
- student
- session
- hall
- attendanceDate = dto.date ?? now()
Save to database
Return AttendanceRecordResponseDTO
{id, studentId, sessionId,
hallId, attendanceDate}
Throw: Invalid student QR code
→ 400
Throw: Attendance already
recorded for this session
→ 400
Throw: Session not found
→ 400
Throw: Hall not found
→ 400
Throw: Hall does not
belong to this session
→ 400
End

6.4 Student Registration with Auto-Parent Creation

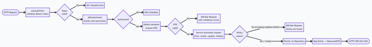
Figure 6.4 – Student Creation Activity Diagram


Start
Client submits
POST /api/students
{userId, parentId?, parentPhoneNumber}
Student already
exists for userId?
No
Load User by userId
User
exists?
Yes
parentId
provided?
No
Search User
by parentPhoneNumber


Yes
Yes
User found by phone?
No
Create new parent User:
- firstName/lastName from student
- email = 'parent_' + student.email
- phone = parentPhoneNumber
- password = phoneNumber (BCrypt)
- forcePasswordChange = true
- forceEmailChange = true
- role = PARENT
Yes
Yes
No
Load Parent by parentId
Save parent User
Use existing User as parent user
Parent found?
Find or create Parent entity for parentUser
Yes
Generate unique QR code token (loop until unique)
No
Build Student entity:
- user, parent, parentPhoneNumber, qrCode
Save Student to database
Return StudentResponseDTO [id, userId, parentId, parentPhoneNumber, qrCode]
Throw: Student already exists for user
Throw: User not found
Throw: Parent not found
End

6.5 Generic CRUD Operation Flow

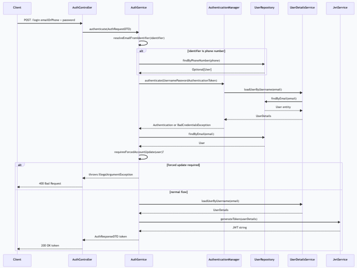
Figure 6.5 – Generic CRUD Activity (applicable to Hall, Session, Parent)


HTTP Request
JwtAuthFilter validates Bearer token
Token valid?
No
401 Unauthorized
Yes
@PreAuthorize checks role/permission
Authorized?
No
403 Forbidden
Yes
@Valid validates request DTO
DTO valid?
No
400 Bad Request with validation errors
Yes
Service processes request (find, create, update, delete)
Entity exists?
No on lookup/update/delete
400 Bad Request: Entity not found
Yes / create
Persist via Repository
Map Entity → ResponseDTO
HTTP 200/201/204

7. Sequence Diagrams

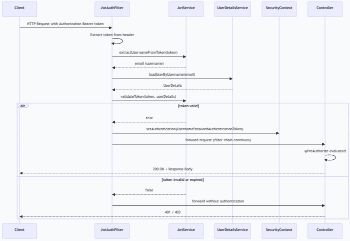
7.1 Login Sequence

Figure 7.1 – Login Request Sequence


Client
AuthController
AuthService
AuthenticationManager
UserRepository
UserDetailsService
JwtService
POST /login emailOrPhone + password
authenticate(AuthRequestDTO)
resolveEmailFromIdentifier(identifier)
alt
[identifier is phone number]
findByPhoneNumber(phone)
Optional[User]
authenticate(UsernamePasswordAuthenticationToken)
loadUserByUsername(email)
findByEmail(email)
User entity
UserDetails
Authentication or BadCredentialsException
findByEmail(email)
User
requiresForcedAccountUpdate(user)?
alt
[forced update required]
throws IllegalArgumentException
400 Bad Request
[normal flow]
loadUserByUsername(email)
UserDetails
generateToken(userDetails)
JWT string
AuthResponseDTO token
200 OK token
Client
AuthController
AuthService
AuthenticationManager
UserRepository
UserDetailsService
JwtService

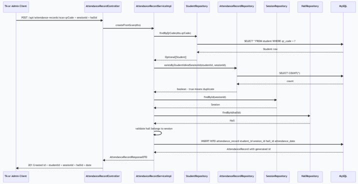
7.2 JWT Filter Chain (Authenticated Request)

Figure 7.2 – JWT Filter Chain Sequence


Client
JwtAuthFilter
JwtService
UserDetailsService
SecurityContext
Controller
HTTP Request with Authorization Bearer token
Extract token from header
extractUsernameFromToken(token)
email (username)
loadUserByUsername(email)
UserDetails
validateToken(token, userDetails)
alt
[token valid]
true
setAuthentication(UsernamePasswordAuthenticationToken)
forward request (filter chain continues)
@PreAuthorize evaluated
200 OK + Response Body
[token invalid or expired]
false
forward without authentication
401 / 403
Client
JwtAuthFilter
JwtService
UserDetailsService
SecurityContext
Controller

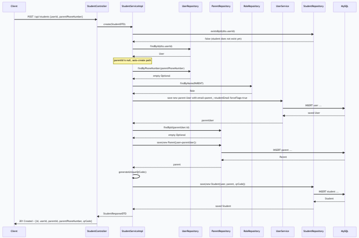
7.3 QR Code Attendance Scan Sequence

Figure 7.3 – QR Attendance Scan Sequence


TA or Admin Client
AttendanceRecordController
AttendanceRecordServiceImpl
StudentRepository
AttendanceRecordRepository
SessionRepository
HallRepository
MySQL
POST /api/attendance-records/scan qrCode + sessionId + hallId
createFromScan(dto)
findByQrCode(dto.qrCode)
SELECT * FROM student WHERE qr_code = ?
Student row
Optional[Student]
existsByStudentIdAndSessionId(studentId, sessionId)
SELECT COUNT(*)
count
boolean - true means duplicate
findById(sessionId)
Session
findById(hallId)
Hall
validate hall belongs to session
INSERT INTO attendance_record student_id session_id hall_id attendance_date
AttendanceRecord with generated id
AttendanceRecordResponseDTO
201 Created id + studentId + sessionId + hallId + date

7.4 Student Creation with Auto-Parent Sequence

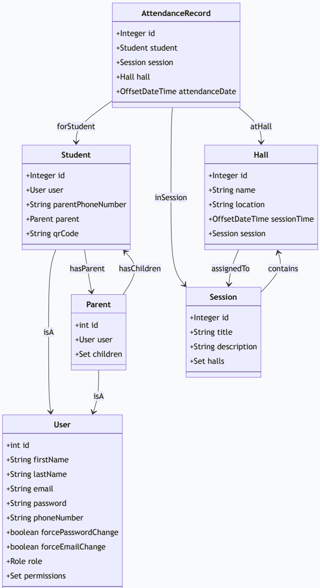
Figure 7.4 – Student Creation (Auto-Parent Path) Sequence

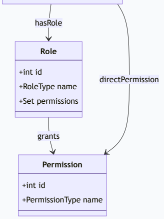

Client
StudentController
StudentServiceImpl
UserRepository
ParentRepository
RoleRepository
UserService
StudentRepository
MySQL
POST /api/students {userId, parentPhoneNumber}
create(StudentDTO)
existsById(dto.userId)
false (student does not exist yet)
findById(dto.userId)
User
parentId is null, auto-create path
findByPhoneNumber(parentPhoneNumber)
empty Optional
findByName(PARENT)
Role
save new parent User with email=parent_+studentEmail forceFlags=true
INSERT user ...
saved User
parentUser
findById(parentUser.id)
empty Optional
save(new Parent[user=parentUser])
INSERT parent ...
Parent
parent
generateUniqueQrCode()
save(new Student[user, parent, qrCode])
INSERT student ...
Student
saved Student
StudentResponseDTO
201 Created + {id, userId, parentId, parentPhoneNumber, qrCode}

8. Class Diagram

The following diagram depicts the full class structure of the system, including entities, DTOs, controllers, services, and repositories, along with their relationships.

Figure 8.1a – Domain Entity Class Diagram


AttendanceRecord
+Integer id
+Student student
+Session session
+Hall hall
+OffsetDateTime attendanceDate
forStudent
atHall
inSession
Student
+Integer id
+User user
+String parentPhoneNumber
+Parent parent
+String qrCode
Hall
+Integer id
+String name
+String location
+OffsetDateTime sessionTime
+Session session
hasParent
hasChildren
assignedTo
contains
Parent
+int id
+User user
+Set children
Session
+Integer id
+String title
+String description
+Set halls
isA
isA
User
+int id
+String firstName
+String lastName
+String email
+String password
+String phoneNumber
+boolean forcePasswordChange
+boolean forceEmailChange
+Role role
+Set permissions


hasRole
Role
+int id
+RoleType name
+Set permissions
directPermission
grants
Permission
+int id
+PermissionType name

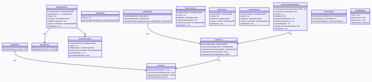
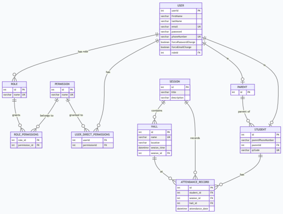


Figure 8.1b – Controller, Service, Security Class Diagram

9. Entity Relationship Diagram (ERD)

The following ERD is derived from JPA `@Entity` annotations, `@JoinColumn` definitions, and `@ManyToOne` / `@OneToOne` / `@ManyToMany` mappings in the source code. The database schema is generated by Hibernate with `ddl-auto=create`.

Figure 9.1 – Entity Relationship Diagram


has role
has
is
is
grants
belongs to
granted to
contains
records
parent of
at
has

USER		
int	userId	PK
varchar	firstName	
varchar	lastName	
varchar	email	UK
varchar	password	
varchar	phoneNumber	UK
boolean	forcePasswordChange	
boolean	forceEmailChange	
int	roleId	FK

ROLE		
int	id	PK
varchar	name	UK

PERMISSION		
int	id	PK
varchar	name	UK

SESSION		
int	id	PK
varchar	title	
varchar	description	

PARENT		
int	id	PK

ROLE_PERMISSIONS		
int	role_id	FK
int	permission_id	FK

USER_DIRECT_PERMISSIONS		
int	userId	FK
int	permissionId	FK

HALL		
int	id	PK
varchar	name	UK
varchar	location	
datetime	session_time	
int	session_id	FK

STUDENT		
int	id	PK
varchar	parentPhoneNumber	
int	parentId	FK
varchar	qrCode	UK

ATTENDANCE_RECORD		
int	id	PK
int	student_id	FK
int	session_id	FK
int	hall_id	FK
datetime	attendance_date	

9.1 Table Descriptions

Table	PK	Notes
user	userId (auto-increment)	Core identity table. Stores credentials and role reference. Unique constraints on email and phoneNumber.
role	id (auto-increment)	Stores role types (ADMIN, STUDENT, PARENT, TEACHING_ASSISTANT). Seeded on startup.
permission	id (auto-increment)	Stores permission types as enum strings. Seeded on startup.
role_permissions	Composite (role_id, permission_id)	Join table for Role ↔ Permission many-to-many.
user_direct_permissions	Composite (userId, permissionId)	Join table for User ↔ Permission direct assignment many-to-many.
parent	id (shared PK from user.userId)	One-to-one with User using @MapsId. No additional columns.
student	id (shared PK from user.userId)	One-to-one with User using @MapsId. Has FK to parent. Unique qrCode column (64 chars).
session	id (auto-increment)	Academic session grouping. Has many halls and attendance records.
hall	id (auto-increment)	Physical room. Optional FK to session. Cascades delete of attendance records.
attendance_record	id (auto-increment)	Junction: student × session × hall + timestamp. Duplicate constraint enforced at service level.

10. API Reference

All API endpoints use **JSON** for request and response bodies unless otherwise noted. Protected endpoints require an `Authorization: Bearer <JWT>` header.

10.1 Authentication Endpoints (Public)

Method	Path	Description	Auth
POST	`/register`	Create a new user account	None
POST	`/login`	Authenticate; receive JWT	None

POST	`/complete-forced-account-update`	Resolve forced update; receive new JWT	None

10.2 User Endpoints

Method	Path	Description	Role
GET	`/api/users`	List all users	ADMIN
GET	`/api/users/me`	Get authenticated user's profile	Any authenticated

10.3 Student Endpoints

Method	Path	Description	Role
POST	`/api/students`	Create student (auto-creates parent if needed)	Authenticated
GET	`/api/students`	List all students	Authenticated
GET	`/api/students/{id}`	Get student by ID	Authenticated
GET	`/api/students/{id}/qr`	Get student QR code as PNG image	Authenticated
GET	`/api/students/parent/{parentId}`	List students by parent ID	Authenticated
PUT	`/api/students/{id}`	Update student record	Authenticated
DELETE	`/api/students/{id}`	Delete student (blocked if has attendance)	Authenticated

10.4 Parent Endpoints

Method	Path	Description	Role
POST	`/api/parents`	Create parent	ADMIN, TA
GET	`/api/parents`	List all parents	ADMIN, TA
GET	`/api/parents/{id}`	Get parent by ID	ADMIN or own PARENT
GET	`/api/parents/{id}/children`	List children of parent	ADMIN or own PARENT
PUT	`/api/parents/{id}`	Update parent	ADMIN, TA, or own PARENT
DELETE	`/api/parents/{id}`	Delete parent	ADMIN, TA

10.5 Hall Endpoints

Method	Path	Description	Role
POST	`/api/halls`	Create hall	ADMIN
GET	`/api/halls`	List all halls	ADMIN, TA
GET	`/api/halls/{id}`	Get hall by ID	ADMIN, TA
PUT	`/api/halls/{id}`	Update hall	ADMIN
DELETE	`/api/halls/{id}`	Delete hall	ADMIN

10.6 Session Endpoints

Method	Path	Description	Role
POST	`/api/sessions`	Create session	ADMIN, TA
GET	`/api/sessions`	List all sessions	ADMIN, TA
GET	`/api/sessions/{id}`	Get session by ID	ADMIN, TA

PUT	`/api/sessions/{id}`	Update session	ADMIN, TA
DELETE	`/api/sessions/{id}`	Delete session (cascade)	ADMIN

10.7 Attendance Record Endpoints

Method	Path	Description	Role
POST	`/api/attendance-records/scan`	Record attendance via QR scan	ADMIN, TA
POST	`/api/attendance-records`	Record attendance manually	ADMIN, TA
GET	`/api/attendance-records`	List all attendance records	ADMIN, TA
GET	`/api/attendance-records/{id}`	Get record by ID	ADMIN, TA
GET	`/api/attendance-records/student/{studentId}`	Get records by student	ADMIN, TA, or own STUDENT
GET	`/api/attendance-records/session/{sessionId}`	Get records by session	ADMIN, TA
GET	`/api/attendance-records/hall/{hallId}`	Get records by hall	ADMIN, TA
PUT	`/api/attendance-records/{id}`	Update attendance record	ADMIN, TA
DELETE	`/api/attendance-records/{id}`	Delete attendance record	ADMIN, TA

11. Module Descriptions

11.1 Package Structure

```
com.lechthemitch.sms
├── config/          DataSeeder — startup data seeding
├── dao/             Spring Data JPA Repository interfaces
├── dto/             Request/Response Record classes (immutable DTOs)
├── entity/          JPA-annotated domain entity classes + enums
├── rest/            REST Controllers + GlobalExceptionHandler
├── security/        JwtAuthFilter, SecurityConfig, UserDetailsImpl
└── service/         Service interfaces + implementations + JwtService +
QrCodeService
```

11.2 config Package

DataSeeder implements `CommandLineRunner` and runs at application startup. It idempotently seeds all Permission enum values, Role definitions with their permission sets, default user accounts (admin, parent, student), and a linked Student with a generated QR code. This ensures the system is functional immediately after launch without manual database population.

11.3 dao Package

All repositories extend `JpaRepository`, providing standard CRUD operations plus custom query methods:

- **UserRepository**: `findByEmail`, `findByPhoneNumber`
- **StudentRepository**: `findByUserId`, `findByParentId`, `findByQrCode`, `existsByQrCode`
- **ParentRepository**: `findByUserId`
- **AttendanceRecordRepository**: `findByStudentId`, `findBySessionId`, `findByHallId`, `existsByStudentIdAndSessionId`
- **RoleRepository**: `findByName(RoleType)`
- **PermissionRepository**: `findByName(PermissionType)`

11.4 security Package

JwtAuthFilter extends `OncePerRequestFilter`. It extracts the Bearer token from the `Authorization` header, validates it via `JwtService`, and populates the `SecurityContext`. It does not block unauthenticated requests itself; the `SecurityFilterChain` handles access control after the filter.

SecurityConfig configures stateless session management, disables CSRF and HTTP Basic, permits public endpoints, and chains the JWT filter before Spring's `UsernamePasswordAuthenticationFilter`. CORS is configured for localhost development.

UserDetailsImpl is a record implementing `UserDetails`, holding a reference to the full `User` entity for downstream access in controllers via `Authentication.getPrincipal()`.

11.5 service Package

JwtService generates HMAC-SHA-256 signed JWT tokens embedding role (prefixed `ROLE_`) and permission claims. It also validates tokens and extracts the username (email) subject claim.

QrCodeService wraps the Google ZXing library to generate QR code images as PNG byte arrays from a given string content at a specified pixel resolution.

AuthService handles registration (encoding passwords, resolving roles and permissions), authentication (supporting phone-number login), and forced account updates with full validation.

StudentServiceImpl manages the complex student-creation logic including auto-parent-account provisioning and unique QR code generation using `SecureRandom`.

11.6 GlobalExceptionHandler

An `@RestControllerAdvice` class that catches `IllegalArgumentException`, Bean Validation failures, and other exceptions, returning structured JSON error responses with appropriate HTTP status codes to clients.

12. Database Design

12.1 Schema Overview

The database is named **SE2** and managed by MySQL 8. Hibernate generates the DDL from entity annotations. The schema uses `ddl-auto=create`, which drops and recreates the schema on each startup — this is appropriate for development but must be changed to `validate` or `update` for production.

12.2 Key Design Decisions

Decision	Rationale from Code
Parent and Student share PKs with User	@MapsId creates a shared primary key with User, modeling an IS-A relationship without table-per-class inheritance overhead
QR Code stored as VARCHAR(64) with UNIQUE	Base64-encoded 24 random bytes = 32 characters; 64-char column provides ample headroom; uniqueness enforced at DB level
Email uniqueness is enforced at DB level and service level	Double protection: @Column(unique=true) plus a pre-check in AuthService.register()
Attendance duplicate prevention is service-level only	The TODO comment in AttendanceRecord acknowledges a lack of DB-level composite unique constraint; currently enforced via existsByStudentIdAndSessionId check
role_permissions and user_direct_permissions as join tables	Many-to-many between Role/User and Permission, allowing flexible permission assignment
Hall CASCADE ALL on AttendanceRecord	Deleting a hall deletes all its attendance records (orphanRemoval=true)
Session CASCADE ALL on Hall and AttendanceRecord	Deleting a session cascades to its halls and attendance records

12.3 Connection Configuration

```
spring.datasource.url     =
${SPRING_DATASOURCE_URL:jdbc:mysql://127.0.0.1:8081/SE2}
spring.datasource.username = ${SPRING_DATASOURCE_USERNAME:myuser}
spring.datasource.password = ${SPRING_DATASOURCE_PASSWORD:secret}
spring.jpa.hibernate.ddl-auto = ${SPRING_JPA_HIBERNATE_DDL_AUTO:create}
```

Docker Compose maps the MySQL container's port 3306 to host port 8081 to avoid conflicts with locally running MySQL instances. Environment variables override defaults for production deployments.

13. Assumptions & Constraints

13.1 Assumptions

- The system assumes a client-side application handles UI rendering; the backend provides only a REST API.
- JWT tokens are transmitted exclusively via the `Authorization: Bearer` HTTP header (no cookies).
- Phone numbers are provided in international format (e.g., +15551234567) and serve as a secondary login identifier.
- Parent accounts auto-created from student registration will have the email prefix `parent_` and must complete the forced account update on first login.
- The QR code token is a security-sensitive value; its uniqueness is guaranteed programmatically using `SecureRandom`.
- Session and Hall data are pre-created by admins before attendance scanning commences.
- The JWT secret in `application.properties` is a development placeholder; production deployments must substitute a strong secret via environment variable.

13.2 Technical Constraints

- The system requires Java 25 and Maven for build.
- MySQL 8 is the only supported database; no JPA dialect abstraction is configured for other DBs.
- CORS is currently restricted to localhost origins; cross-origin production access requires configuration change.
- `ddl-auto=create` destroys data on restart; this must be changed before production deployment.
- The JWT expiration is hardcoded to 36,000,000 ms (10 hours); configurable via properties but not user-tunable.
- No refresh token mechanism exists; expiry requires re-login.
- Attendance duplicate prevention (student + session) is enforced at service level only, not by a database unique constraint — a race condition could theoretically allow duplicates under concurrent load.
- The `ASSIGN_STUDENT_TO_HALL` and `VIEW_STUDENTS_ASSIGNED_TO_HALL` permissions are defined in the enum but not yet wired to any `@PreAuthorize` rule in the current codebase.

13.3 Known Limitations & TODOs (from Source Code)

> The source code contains the following developer TODOs which represent acknowledged technical debt:
>
> - **AttendanceRecord.java**: "This is a terrible way of implementing this, should use lists and advanced hibernate mappings later or Foreign Keys." — a database-level unique constraint on (student_id, session_id) is missing.
> - **SecurityConfig.java**: "TODO Understand what is that" — on the AuthenticationProvider bean; suggests incomplete understanding of the DaoAuthenticationProvider configuration.

14. Glossary

Term	Definition
JWT	JSON Web Token — a compact, URL-safe means of representing claims transferred between two parties, signed with HMAC-SHA-256.
BCrypt	A password-hashing function used to securely store passwords with adaptive computational cost.
RBAC	Role-Based Access Control — a method of restricting system access based on the roles of individual users.
DTO	Data Transfer Object — a plain object used to carry data between process layers, separate from domain entities.
QR Code	Quick Response Code — a matrix barcode used here as a unique student identifier for attendance scanning.
JPA	Jakarta Persistence API — a specification for ORM in Java; implemented here by Hibernate.
Spring Security	The authentication and authorisation framework used to secure the REST API.
Session	Within this system, an academic class session or lecture grouping, not an HTTP session.
Hall	A physical classroom or venue assigned to a session, used for locating attendance.
Forced Account Update	A system state where a user must change their email and password before accessing the system — applied to auto-created parent accounts.
DataSeeder	A Spring CommandLineRunner that populates the database with initial essential data at application startup.
ZXing	Zebra Crossing — an open-source multi-format 1D/2D barcode image processing library from Google, used for QR code generation.
CORS	Cross-Origin Resource Sharing — a mechanism that allows restricted resources on a web page to be requested from another domain.
MapsId	A JPA annotation that maps an entity's primary key to a foreign key, creating a shared-primary-key (IS-A) relationship.

Software Requirements Specification — Student Management System

Generated from static source code analysis of SE2-main.zip | Version 1.0 | May 2026

This document was derived entirely from the submitted Spring Boot source code. All diagrams reflect actual implemented logic.